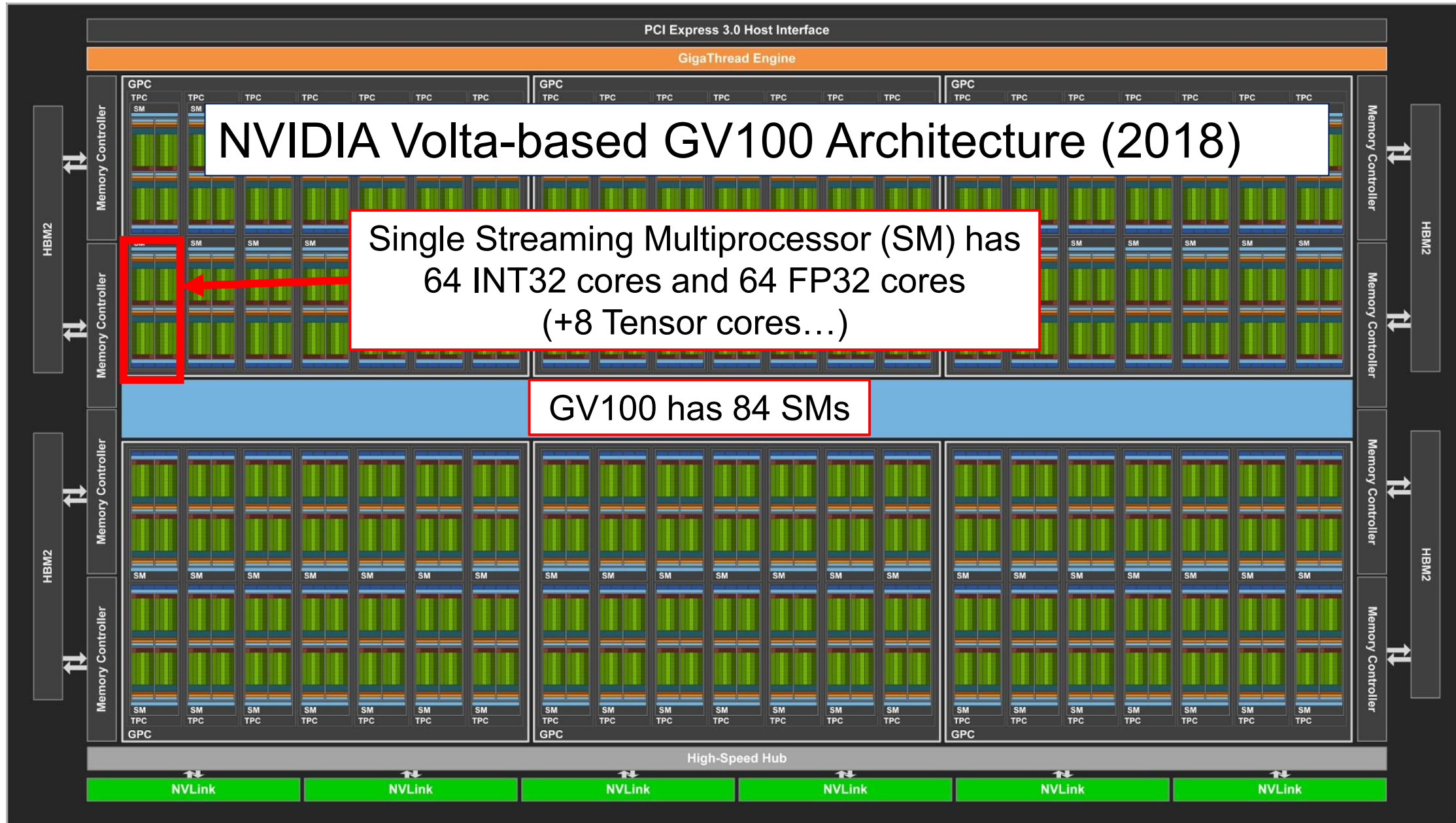




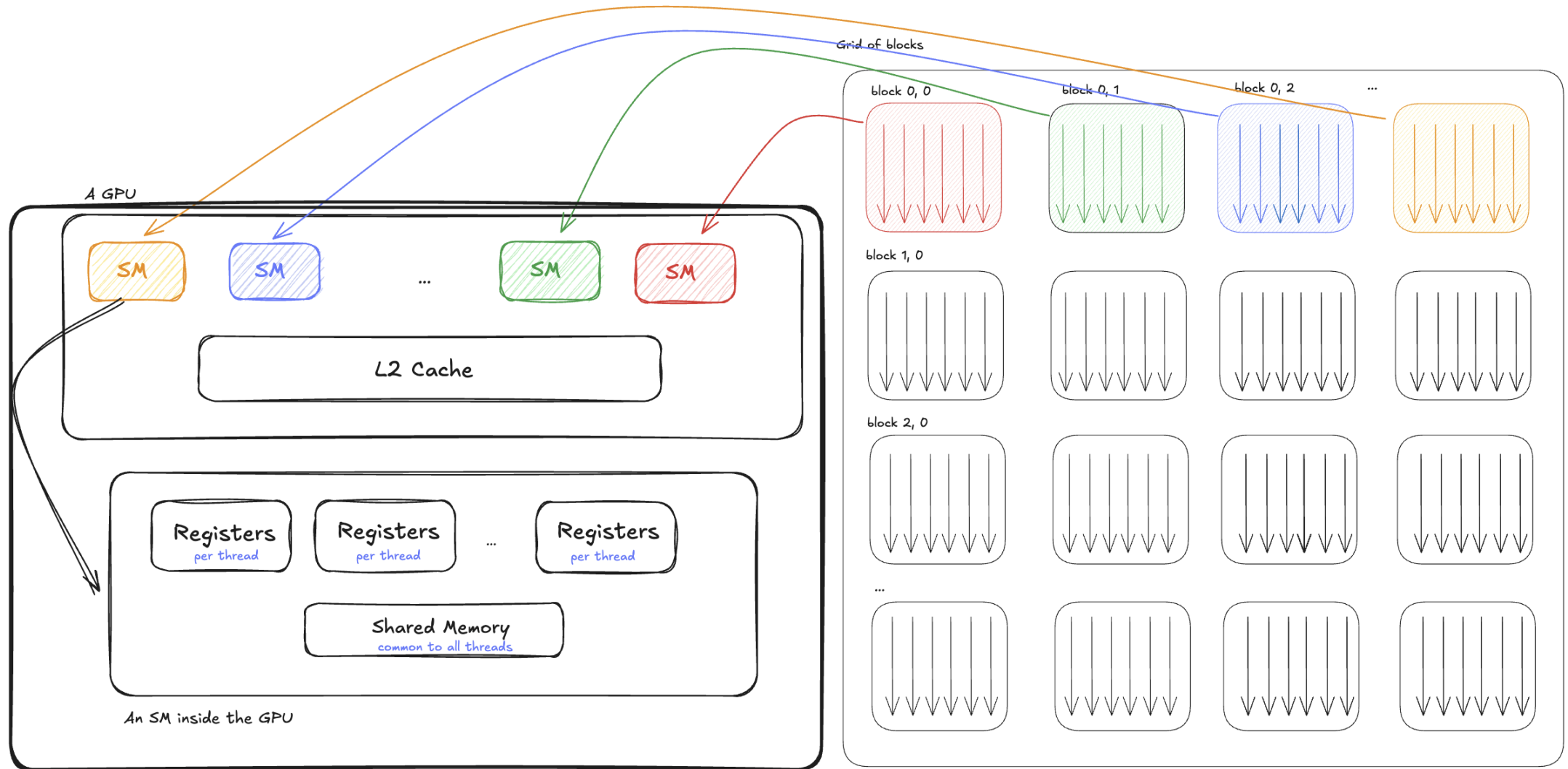
Large Language Models: A Hands-on Approach

GPU-Programming

GPU

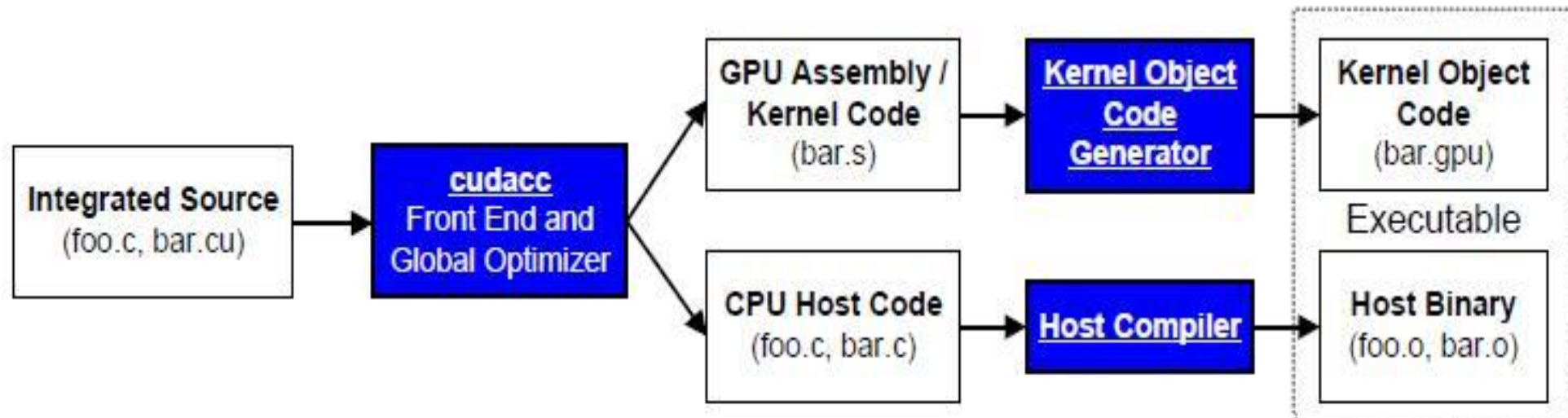


GPU



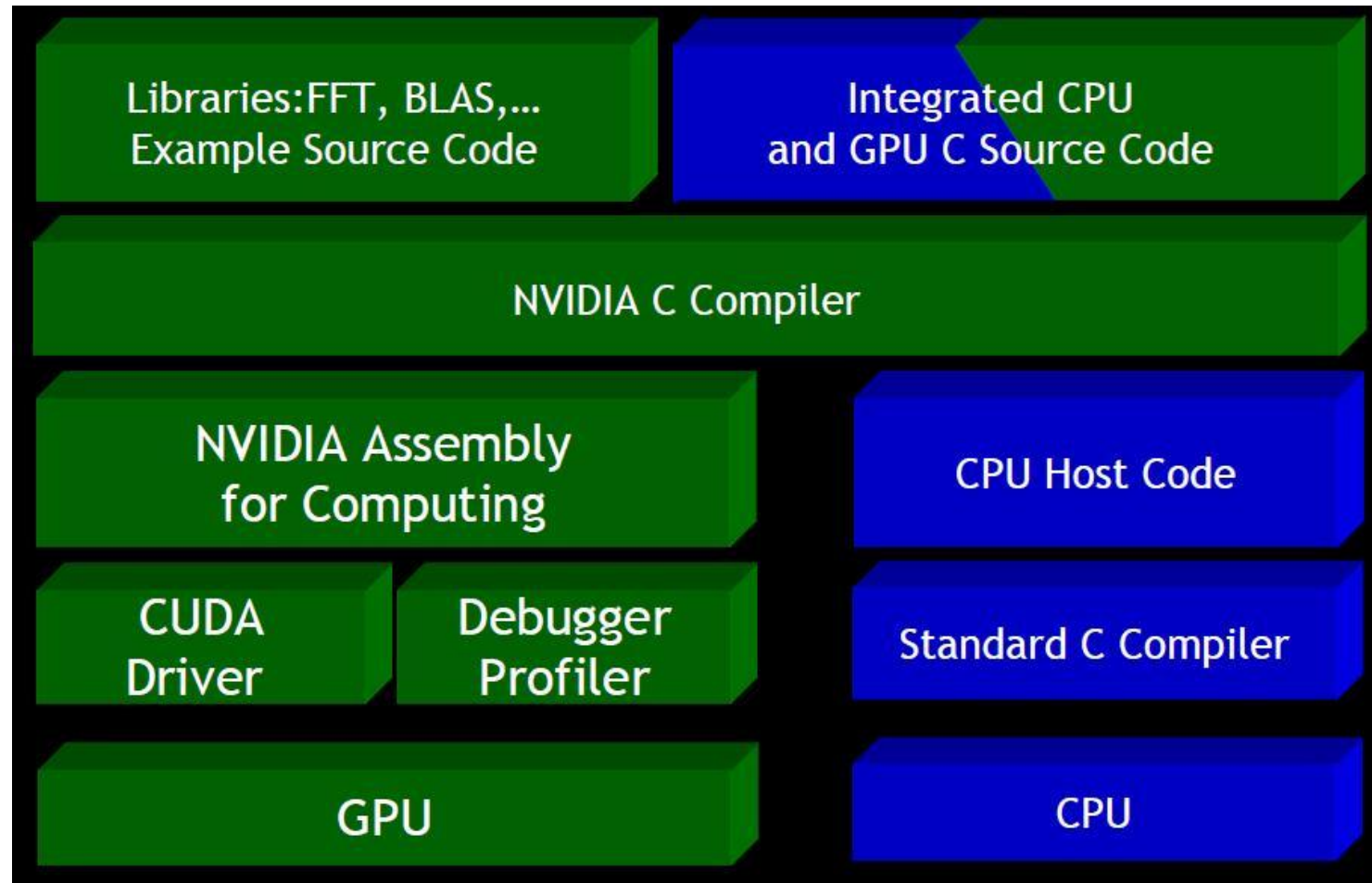
CUDA Compilation

- As a programming model, CUDA is a set of extensions to ANSI C
- **CPU code** is compiled by the host C compiler and the **GPU code** (kernel) is compiled by the CUDA compiler. Separate binaries are produced

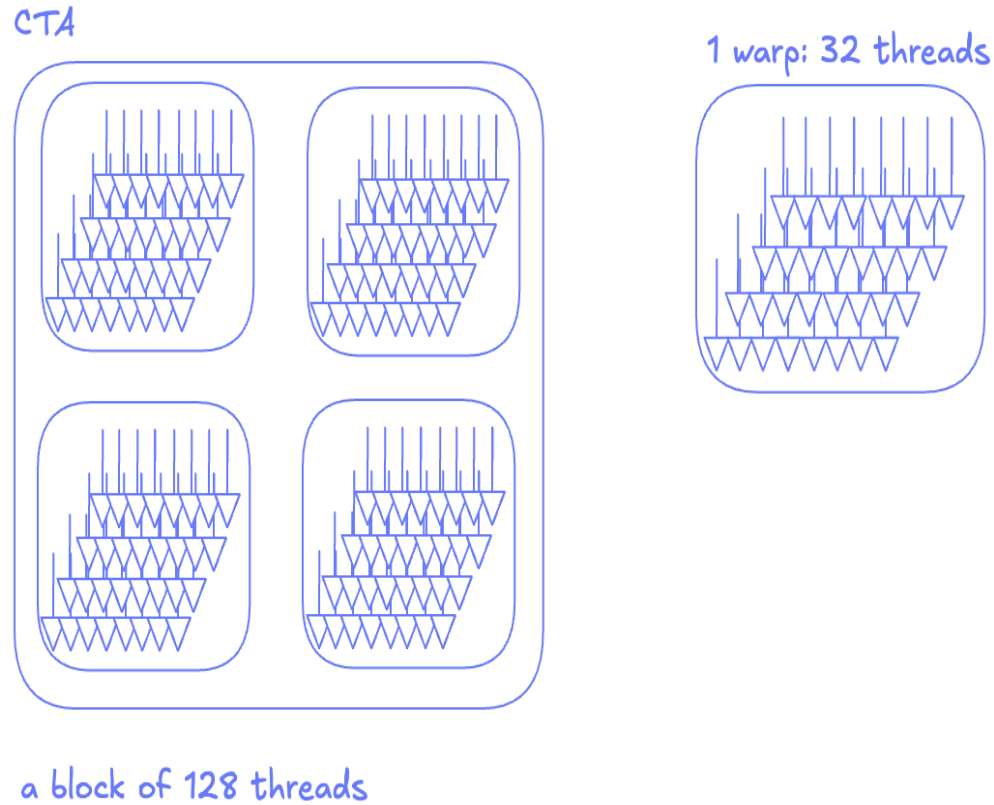


```
$ nvcc -o executable source.cu
```

CUDA Stack



Thread Hierarchy

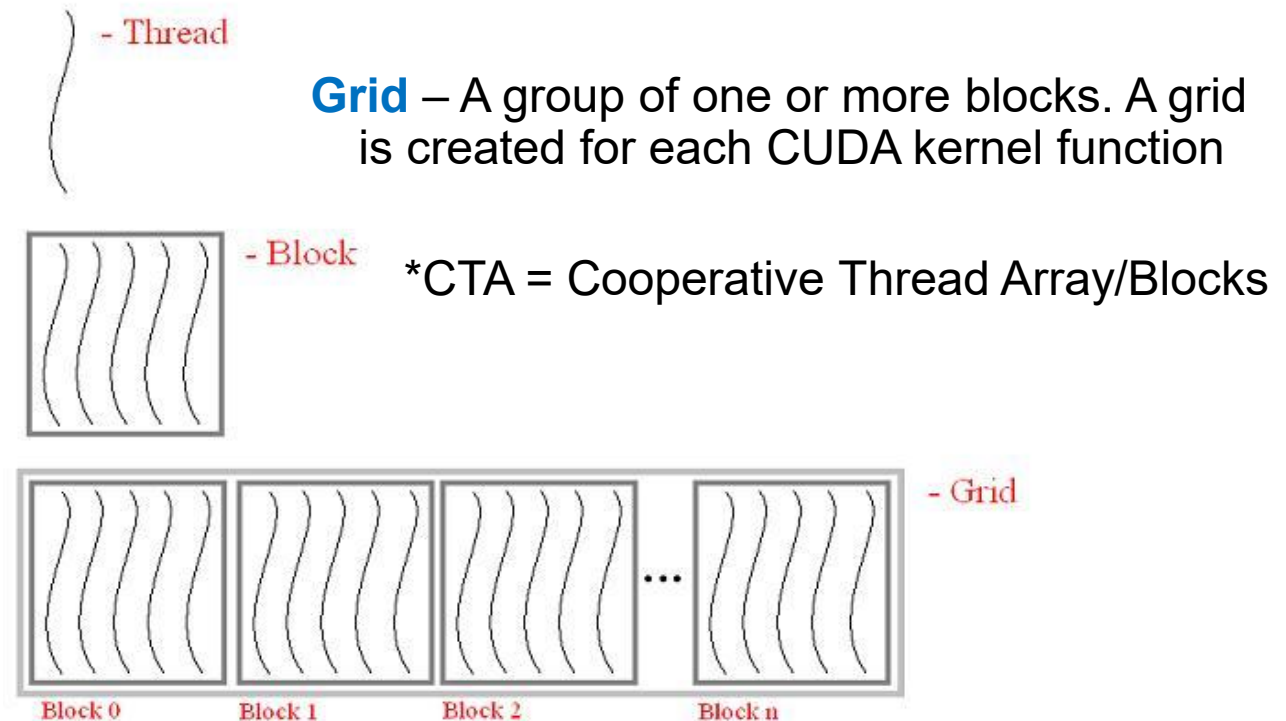


Thread – Distributed by the CUDA runtime (identified by **threadIdx**)

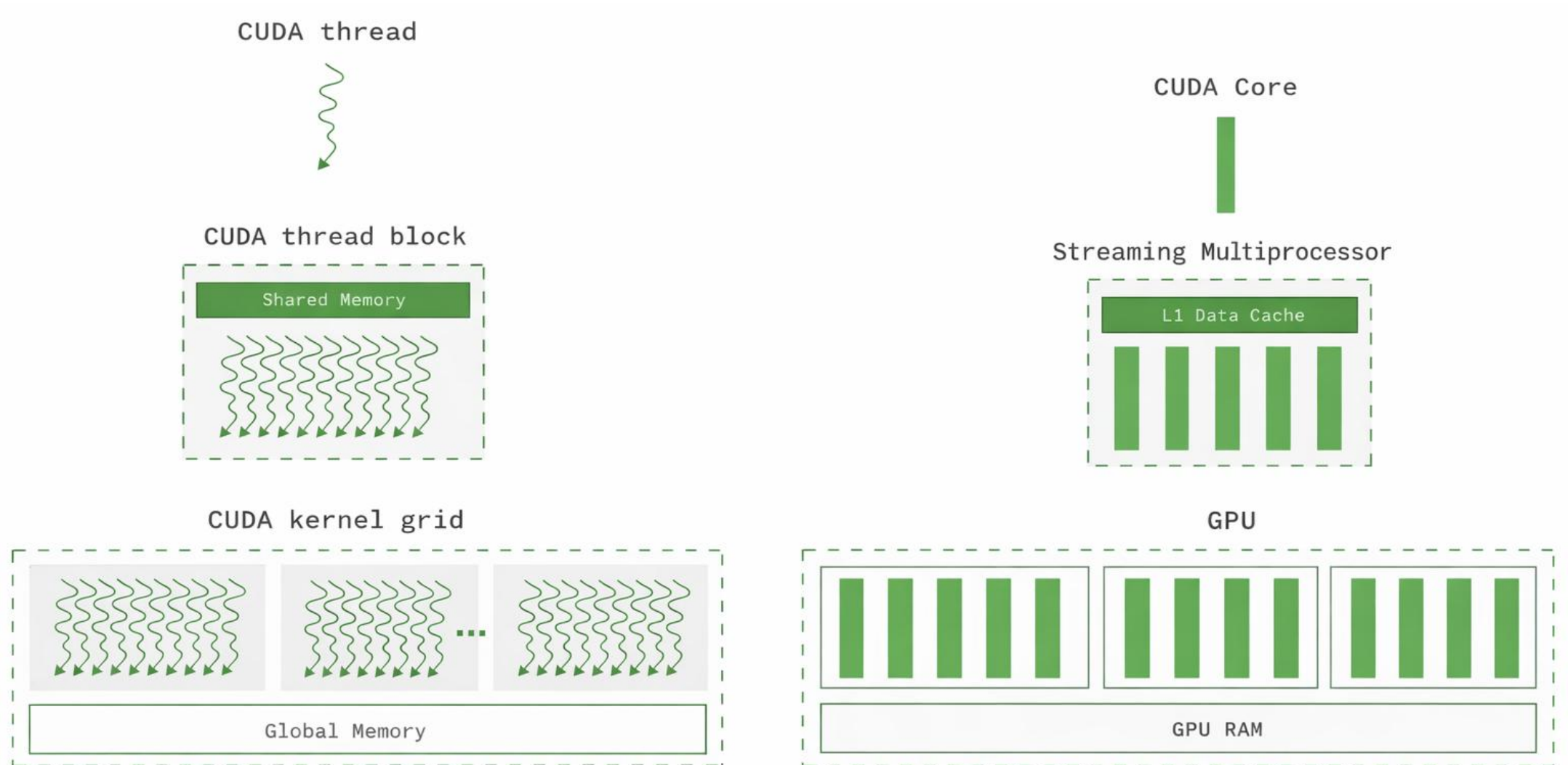
Warp – A scheduling unit of up to 32 threads

Block – A user-defined group of 1 to 512 threads. (identified by **blockIdx**)

Grid – A group of one or more blocks. A grid is created for each CUDA kernel function

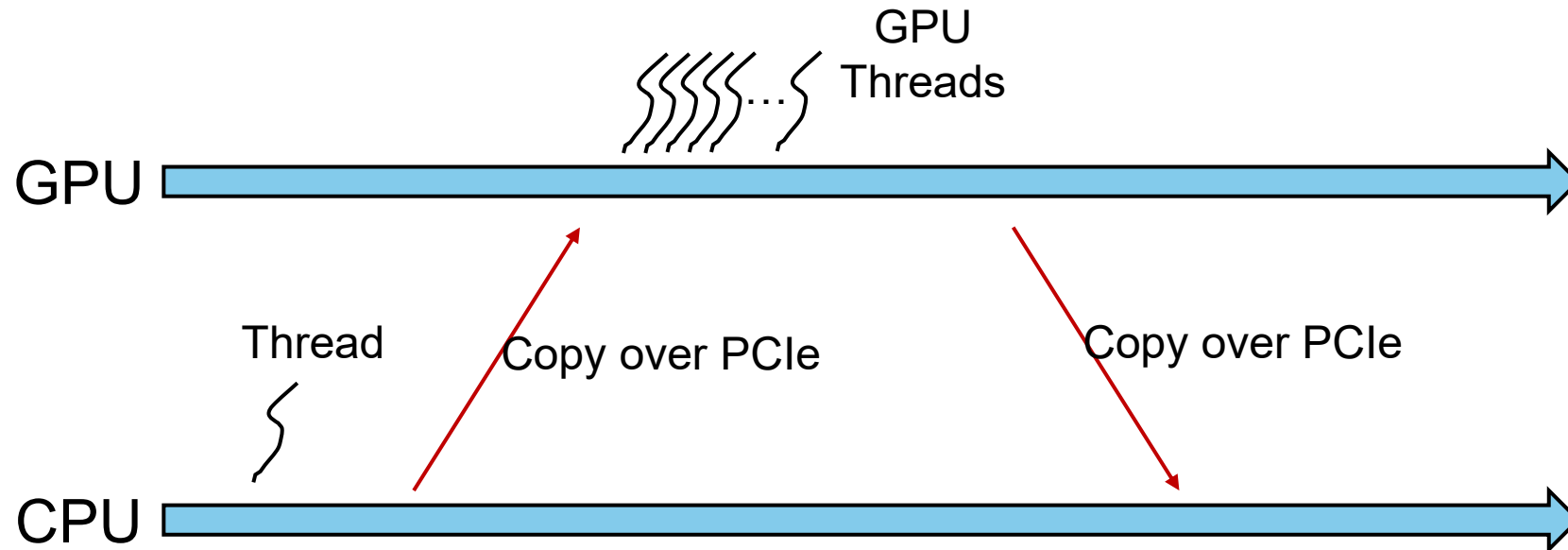


Thread Hierarchy



Massively Parallel Architecture For Massively Parallel Workloads!

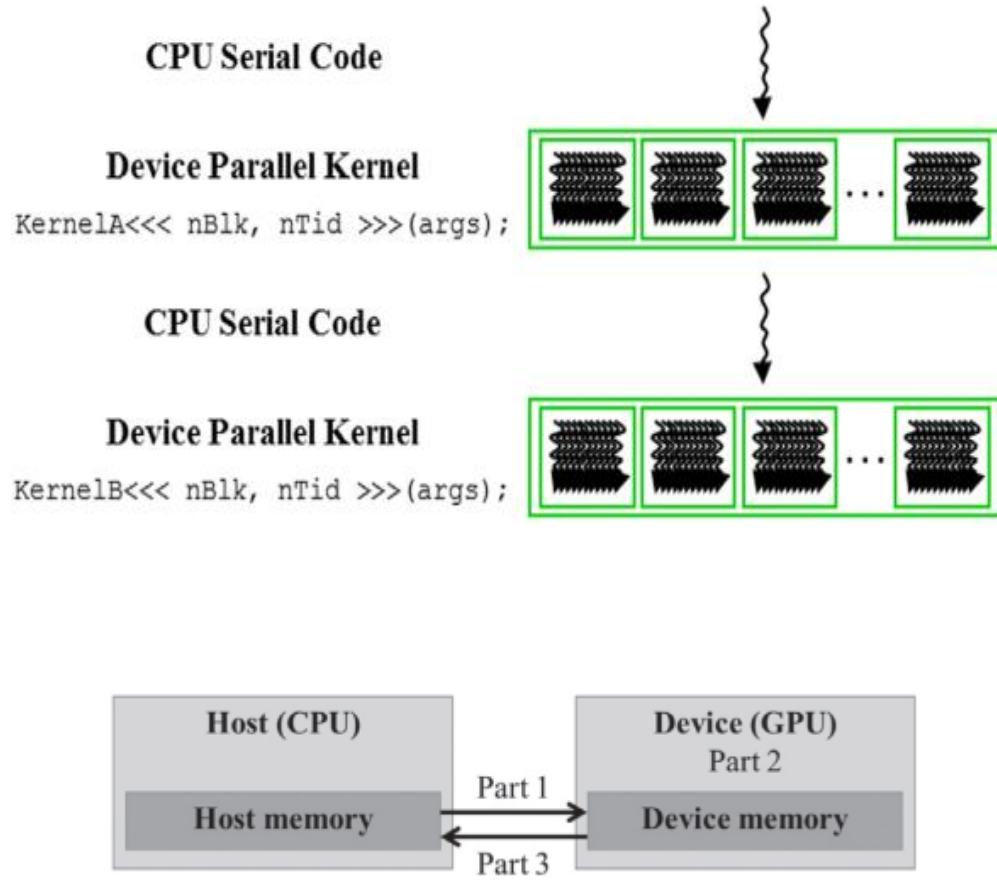
- NVIDIA CUDA (Compute Uniform Device Architecture) – 2007
 - A way to run custom programs on the massively parallel architecture!
- Both platforms expose synchronous execution of a massive number of threads



CUDA Execution Abstraction

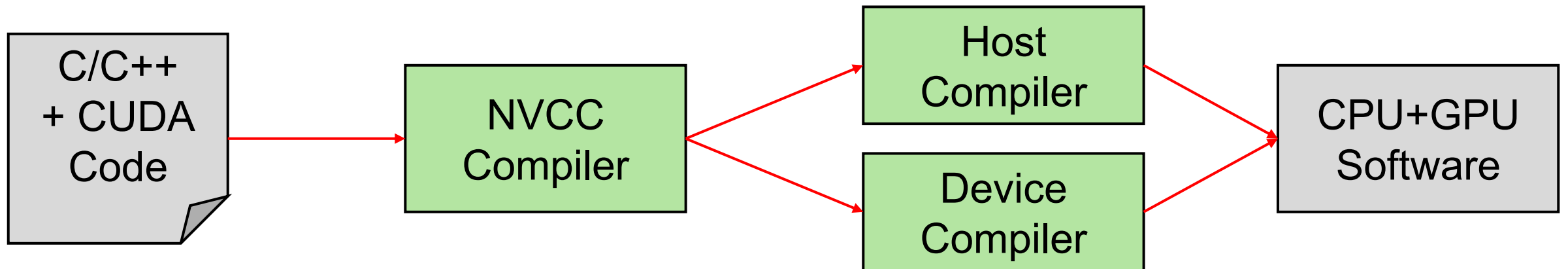
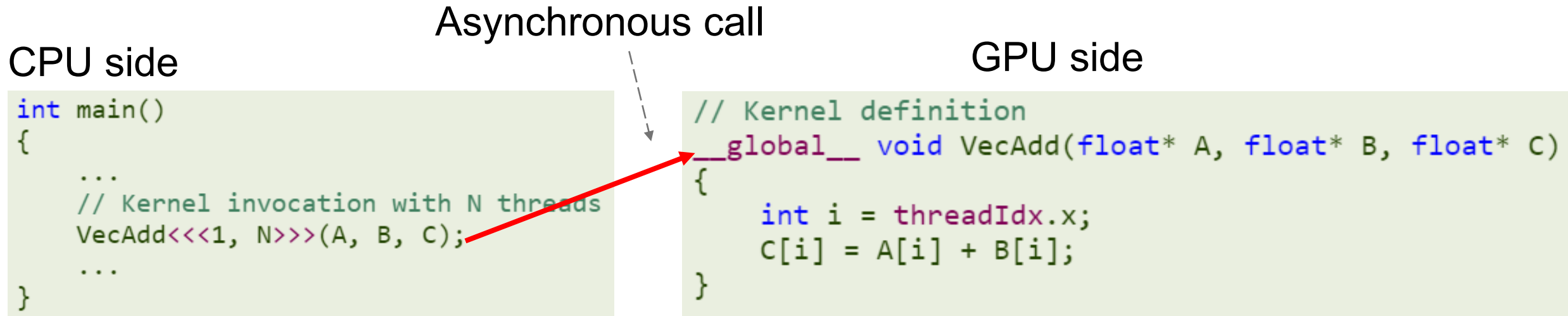
- Block: Multi-dimensional array of threads
 - 1D, 2D, or 3D
 - Threads in a block can synchronize among themselves
 - Threads in a block can access shared memory
 - CUDA (Thread, Block) $\sim$ (Work item, Work group)
- Grid: Multi-dimensional array of blocks
 - 1D or 2D
 - Blocks in a grid can run in parallel, or sequentially
- Kernel execution issued in grid units
- Limited recursion (depth limit of 24 as of now)

CUDA Execution



```
void vecAdd(float* A, float* B, float* C, int n) {  
    int size = n* sizeof(float);  
    float *d_A *d_B, *d_C;  
  
    // Part 1: Allocate device memory for A, B, and C  
    // Copy A and B to device memory  
    ...  
  
    // Part 2: Call kernel - to launch a grid of threads  
    // to perform the actual vector addition  
    ...  
  
    // Part 3: Copy C from the device memory  
    // Free device vectors  
    ...  
}
```

Simple CUDA Example



Simple CUDA Example

```
int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
}
```

1 block

N threads per block

Should wait for the kernel to finish

__global__:
In GPU, called from host/GPU

__device__:
In GPU, called from GPU

__host__:
In host, called from host

```
// Compute vector sum C = A + B
// Each thread performs one pair-wise addition
__global__
void vecAddKernel(float* A, float* B, float* C, int n) {
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}
```

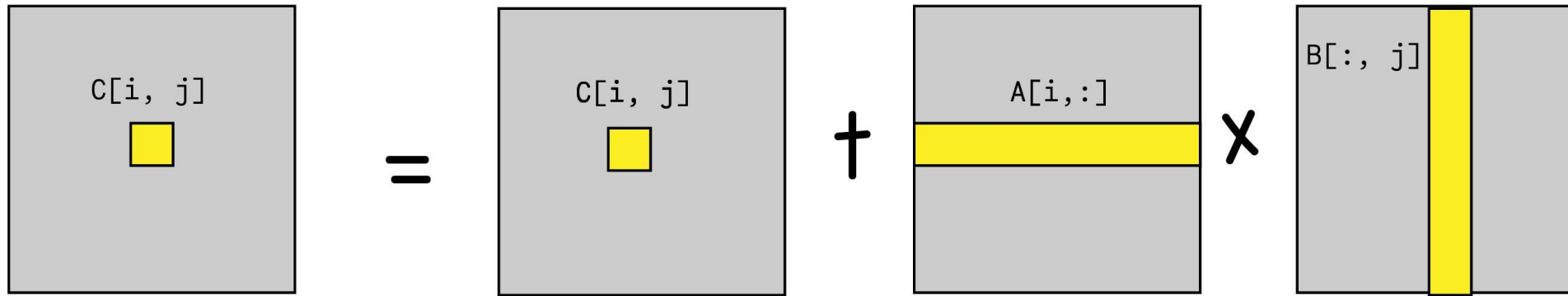
N instances of VecAdd spawned in GPU

Which of N threads am I?
See also: blockIdx

Only void allowed

One function
can be both

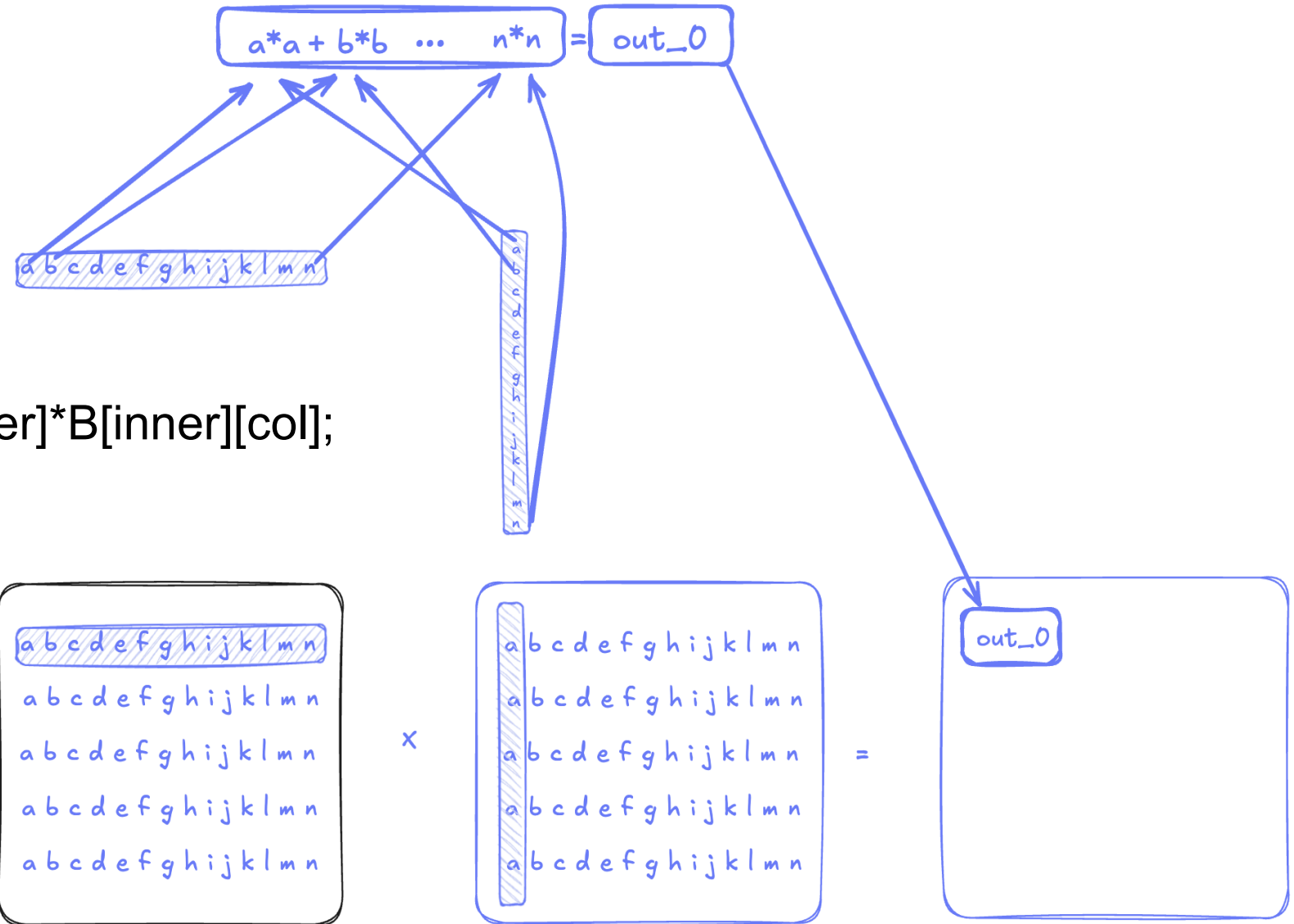
Matrix-Matrix Multiplication



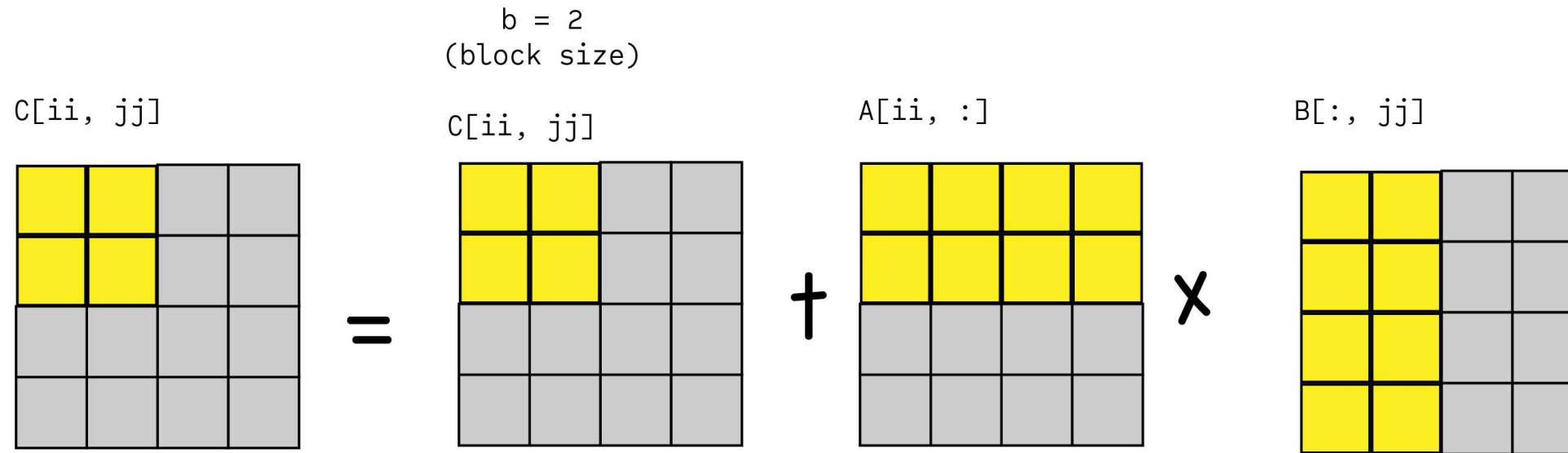
```
// C = C + A * B
for i = 1 to n:
    // read row i of A into fast memory
    for j = 1 to n:
        // read C[i, j] into fast memory
        // read col j of B into fast memory
        for k = 1 to n:
            C[i, j] += A[i, k] * B[k, j]
        // write C[i, j] back to slow memory
```

Matrix-Matrix Multiplication

```
for row in range(M)
  for col in range(N)
    for inner in range(K):
      result[row][col] += A[row][inner] * B[inner][col];
```



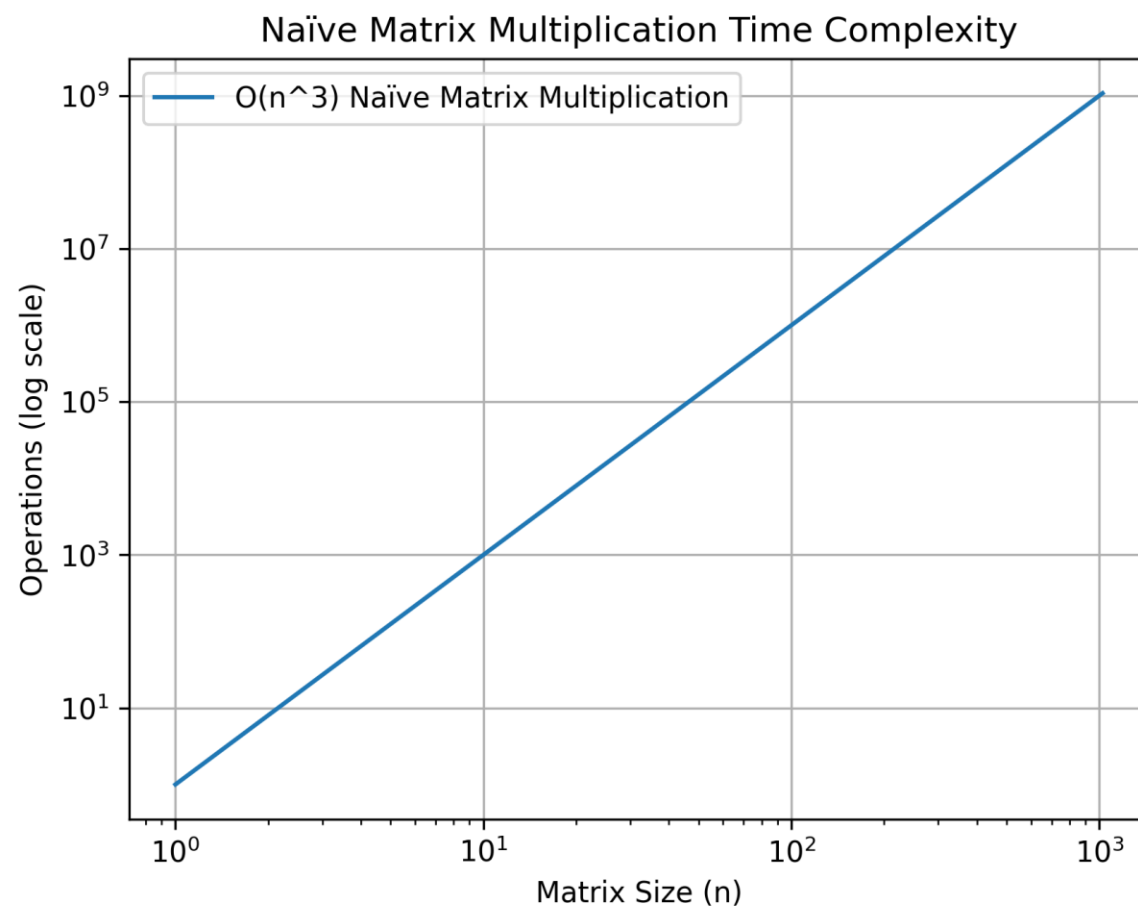
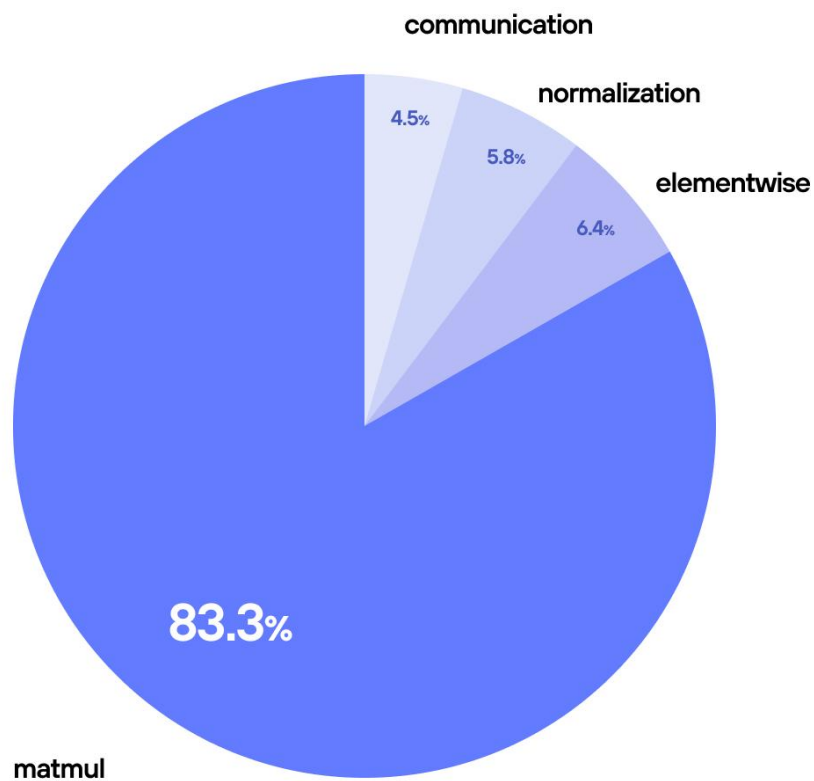
Matrix-Matrix Multiplication (Block)



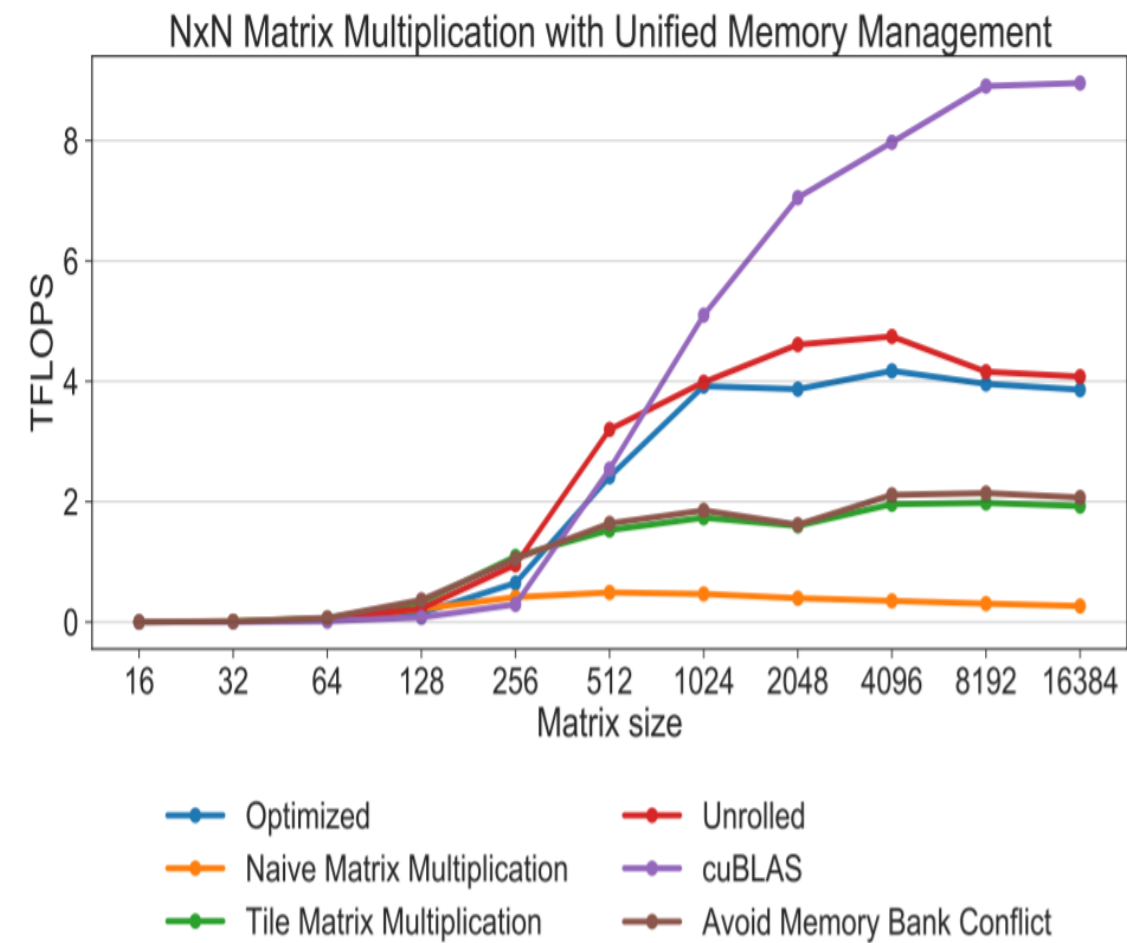
ii, jj, kk denote block indices while i, j, k denote element indices

```
// C = C + A * B
// b = n / N (where b is the block size)
for ii = 1 to N:
  for jj = 1 to N:
    for kk = 1 to N:
      C[ii, jj] += A[ii, kk] * B[kk, jj]
```

Matrix-Matrix Multiplication in LLM's



Matrix Multiplication Performance



Curve	Implementation	Parallelism Level
Naïve	Global memory only	Thread
Tile	Shared memory	Thread block
Optimized	Coalesced access	Block
Unrolled	ILP	Warp
Avoid Bank Conflict	SMEM optimized	Warp
cuBLAS	Tensor Core WMMA	Warp MMA

Results from NVIDIA P100

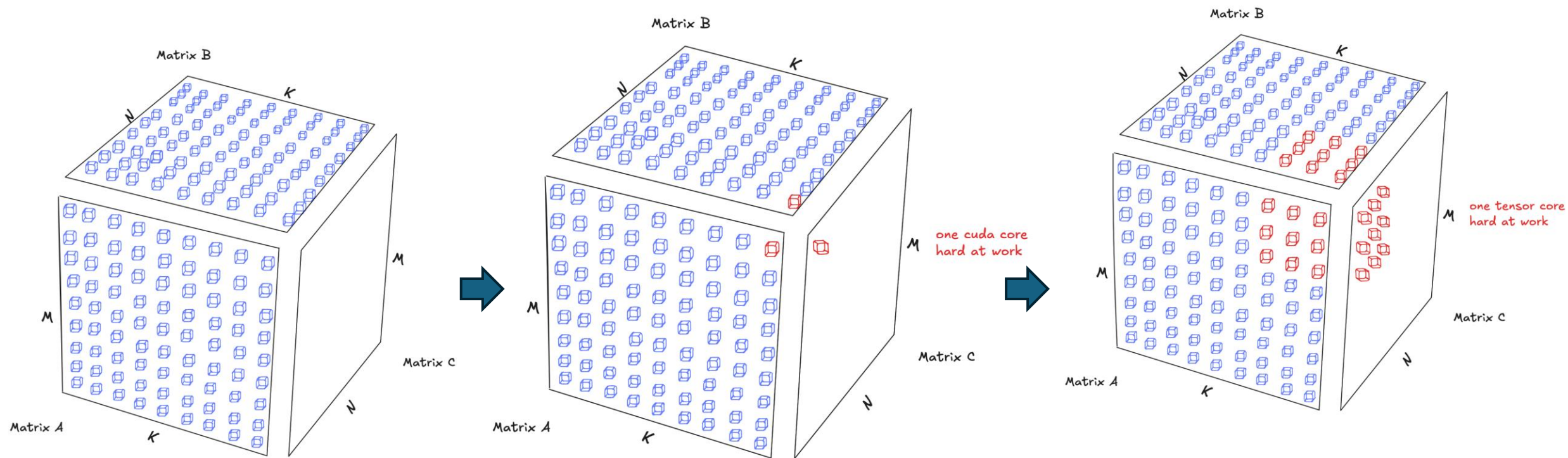
Role of Matrix Multiplication in LLMs

- **Embedding Layers:** Converts input tokens into high-dimensional vectors using learned embeddings stored in matrices.
- **Attention Mechanisms:** Computes relationships between words using query, key, and value matrices in self-attention layers.
- **Feedforward Networks:** Apply transformations through dense layers, which are essentially large matrix multiplications.
- **Output Projections:** Maps hidden states back to vocabulary space for token prediction.

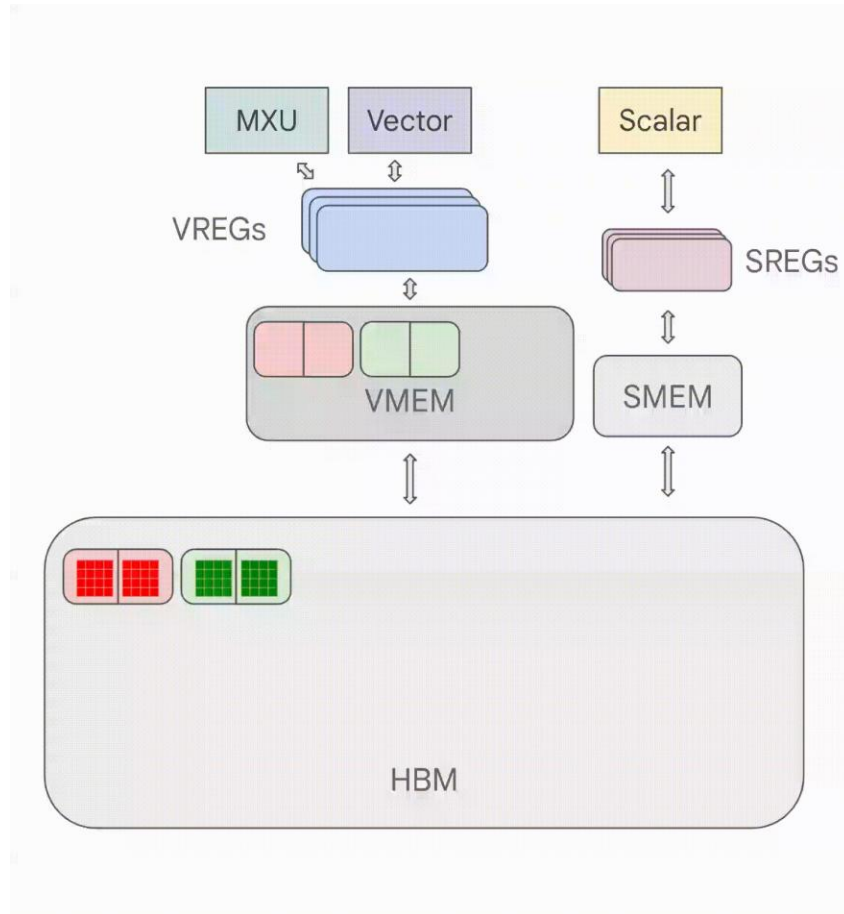
How Matrix Multiplication is Accelerated

- **Parallel Processing:** Thousands of CUDA cores enable simultaneous computation of matrix blocks.
- **Optimized Libraries:** CUDA-accelerated libraries like cuBLAS and cuDNN provide highly tuned matrix multiplication kernels.
- **Tensor Cores:** Specialized units in NVIDIA GPUs (A100, H100, L40S) that perform mixed-precision matrix operations with extreme efficiency.
- **High Memory Bandwidth:** GPUs like the H100 NVL provide massive memory bandwidth (up to 3TB/s) to feed data quickly to compute units.

Matrix Multiplication With TPU



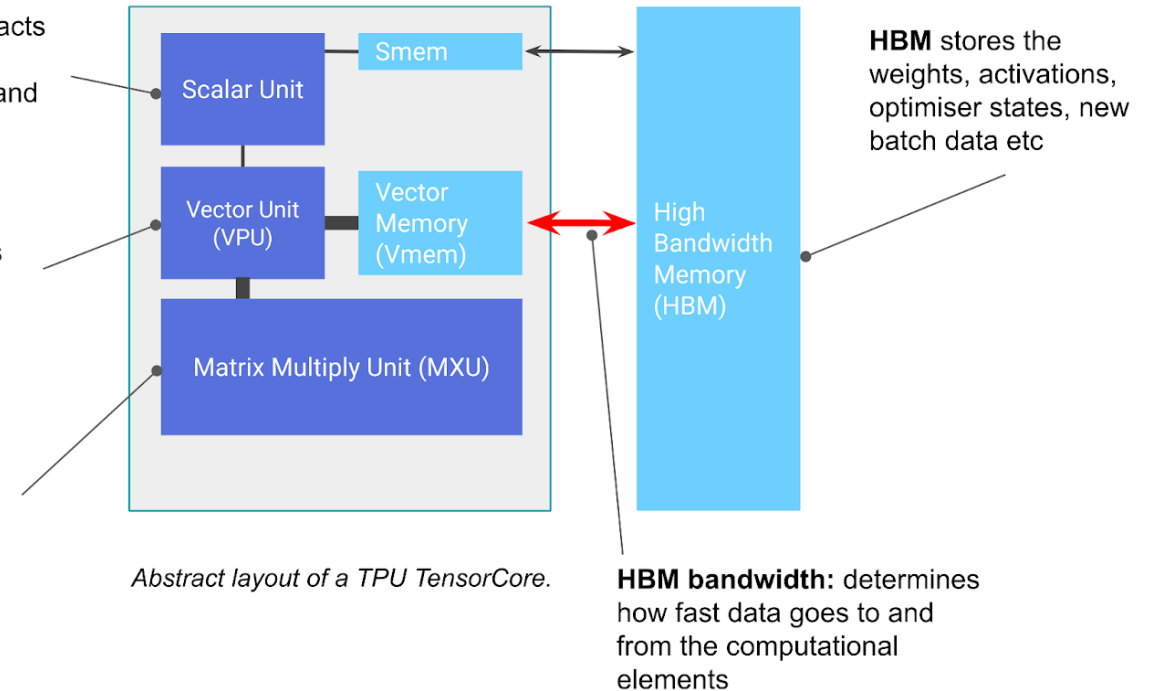
Matrix Multiplication With TPU



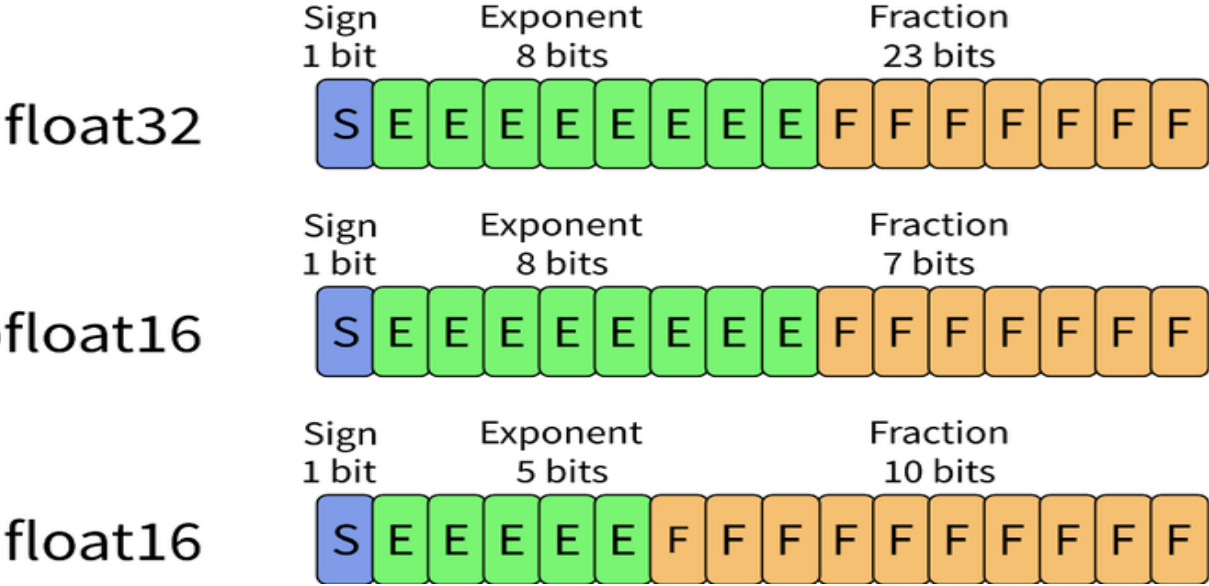
The **Scalar Unit** sort of acts like a CPU 'dispatching' instructions to the VPU and MXU

The **VPU** performs elementwise operations (e.g. activations), loads data into the MXU

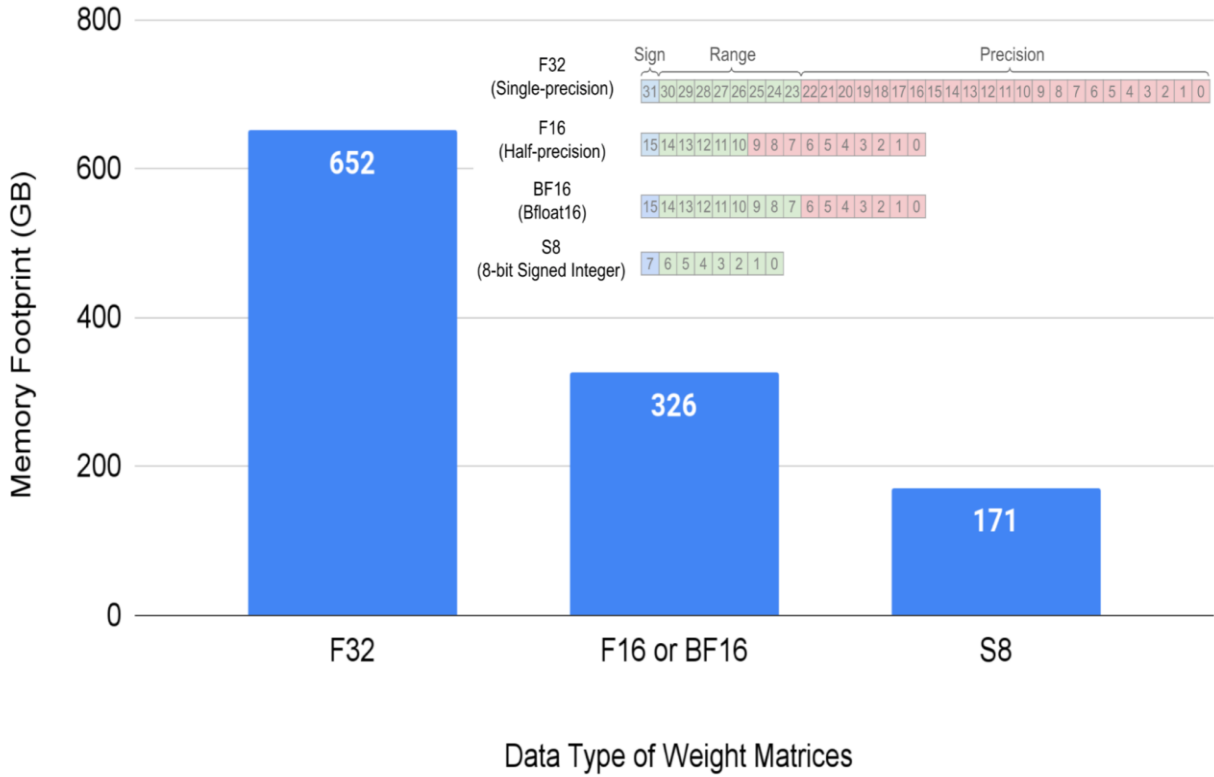
The **MXU** performs matrix multiplications - and is therefore our driver of chip FLOP/s.



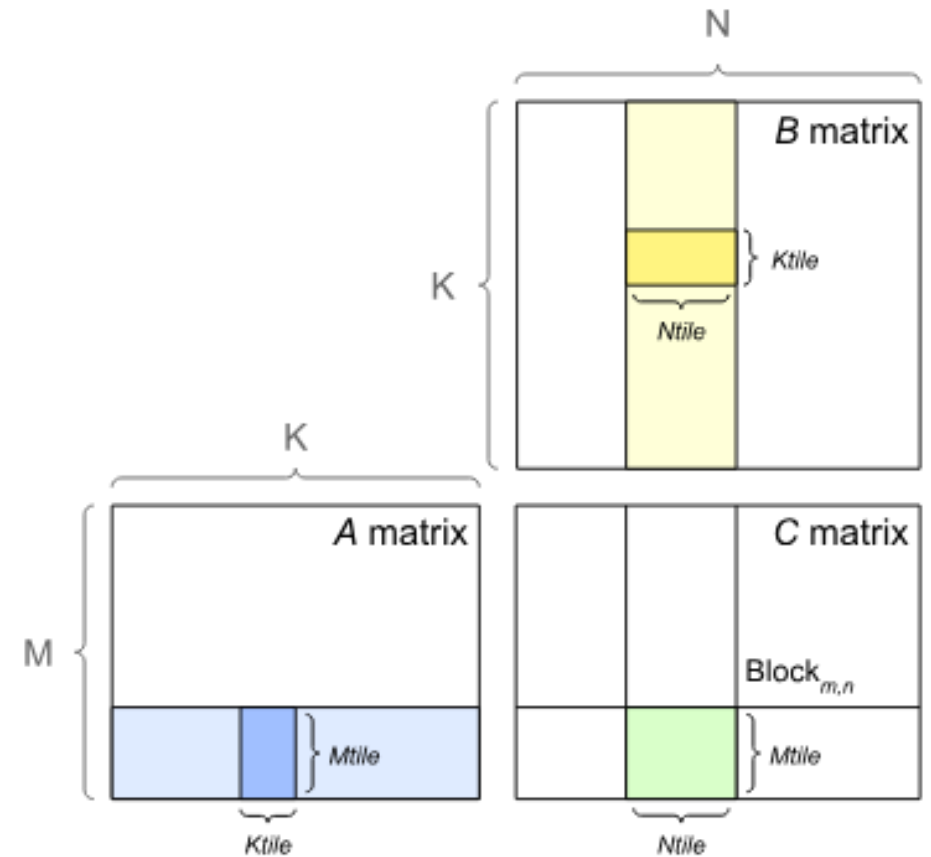
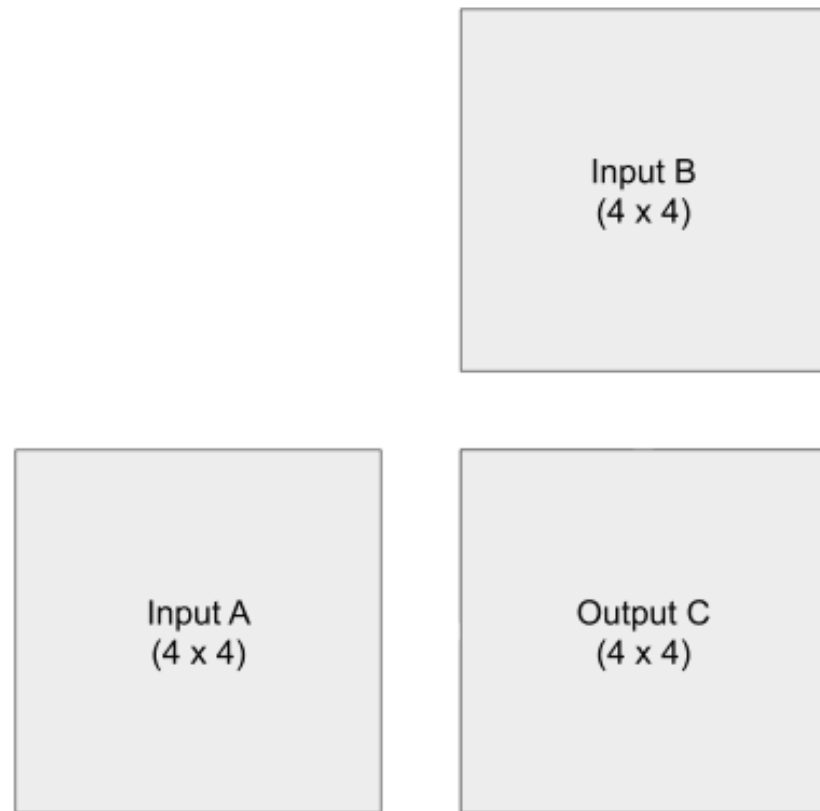
Mixed Precision



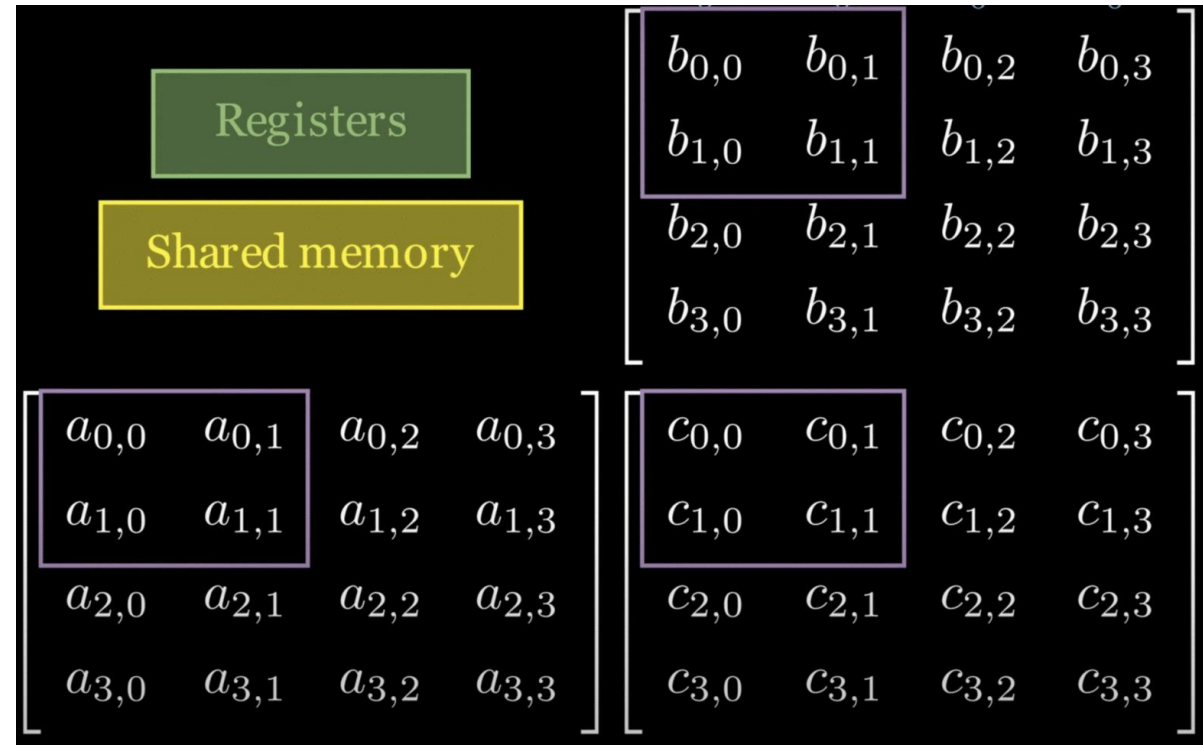
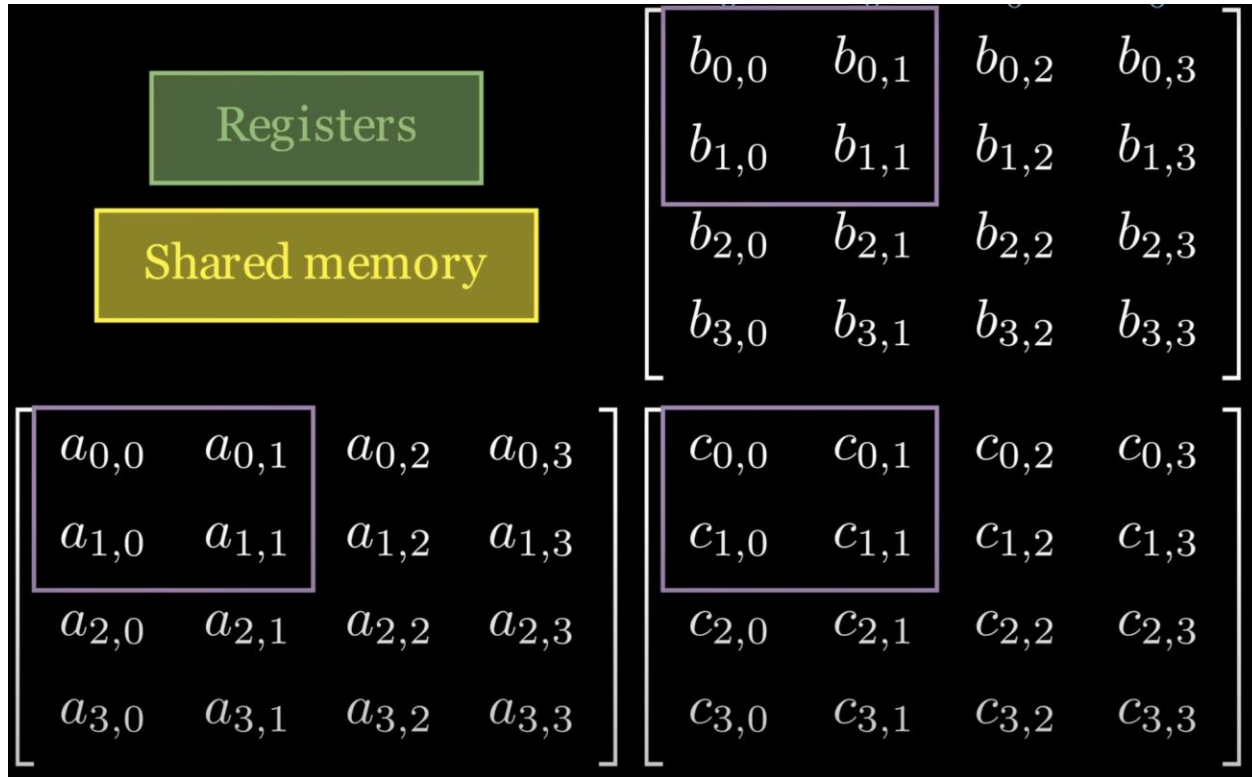
Impact of Data Type on LLM Memory Footprint



Matrix Tiling

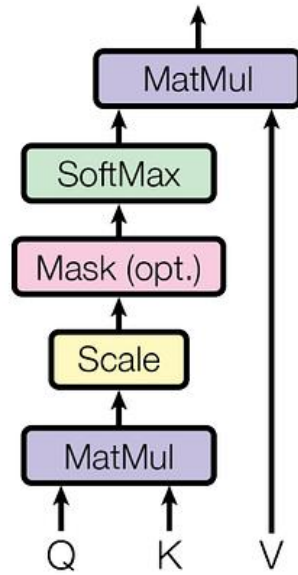


Matrix Tiling

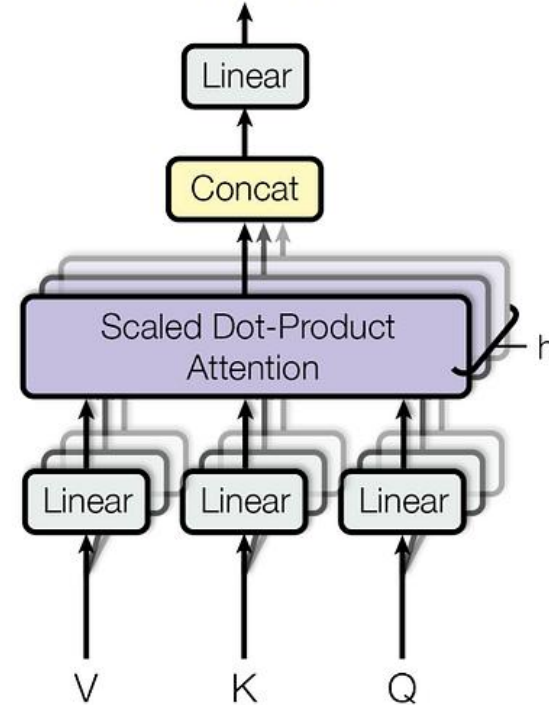


Flash Attention

Scaled Dot-Product Attention

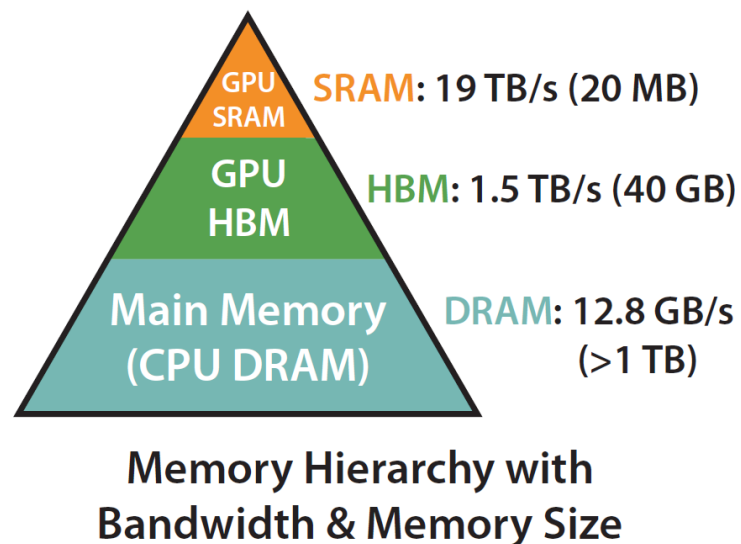


Multi-Head Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Flash Attention

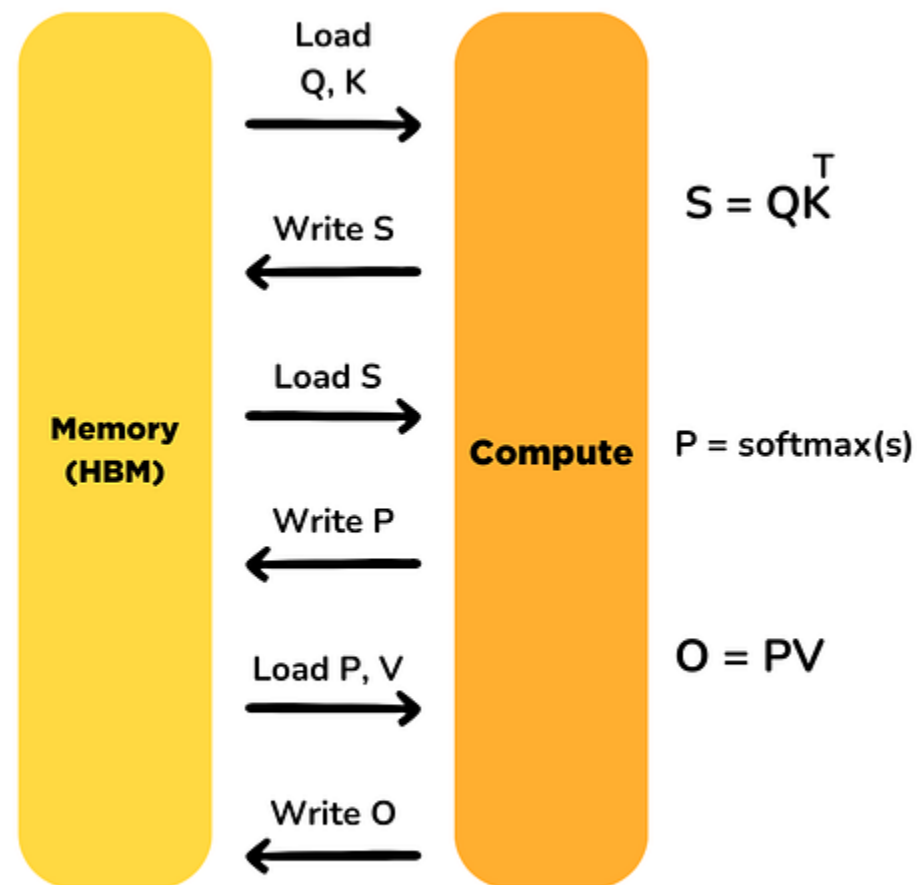


Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Standard Attention Implementation

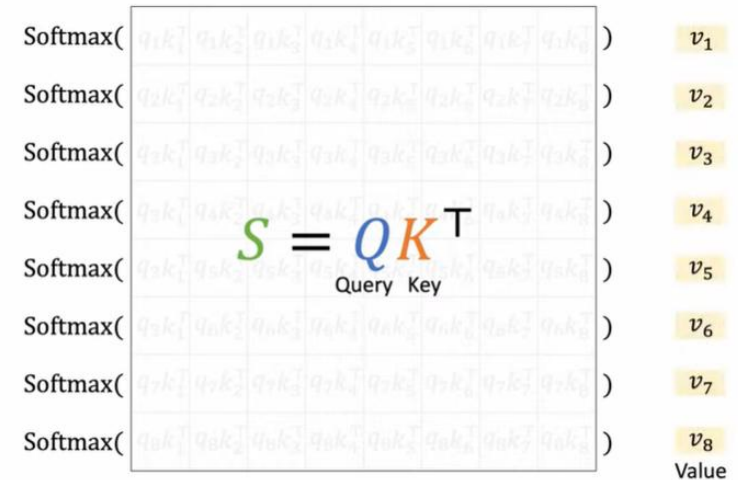


Flash Attention

- **Reusing** data loaded in shared memory for multiple steps. A prime example of this is tiled matrix multiplication
- **Fusing** multiple operations in a single kernel (since every kernel launch implies moving data from DRAM to SRAM).
- **Online softmax**

$$y_i = \frac{\exp(x_i - m)}{\sum_{j=1}^N \exp(x_j - m)}, \quad m = \max_j(x_j)$$

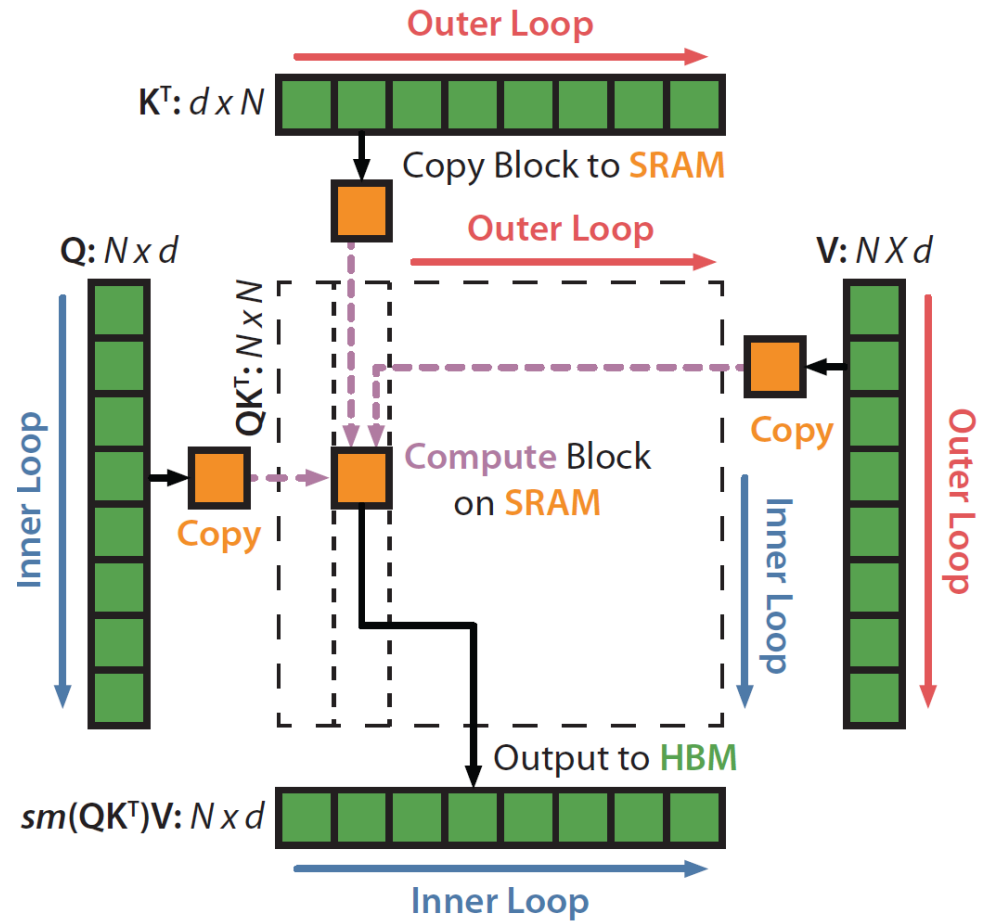
$$m_i \leftarrow \max(m_i, m_{\text{tile}}^{(t)})$$



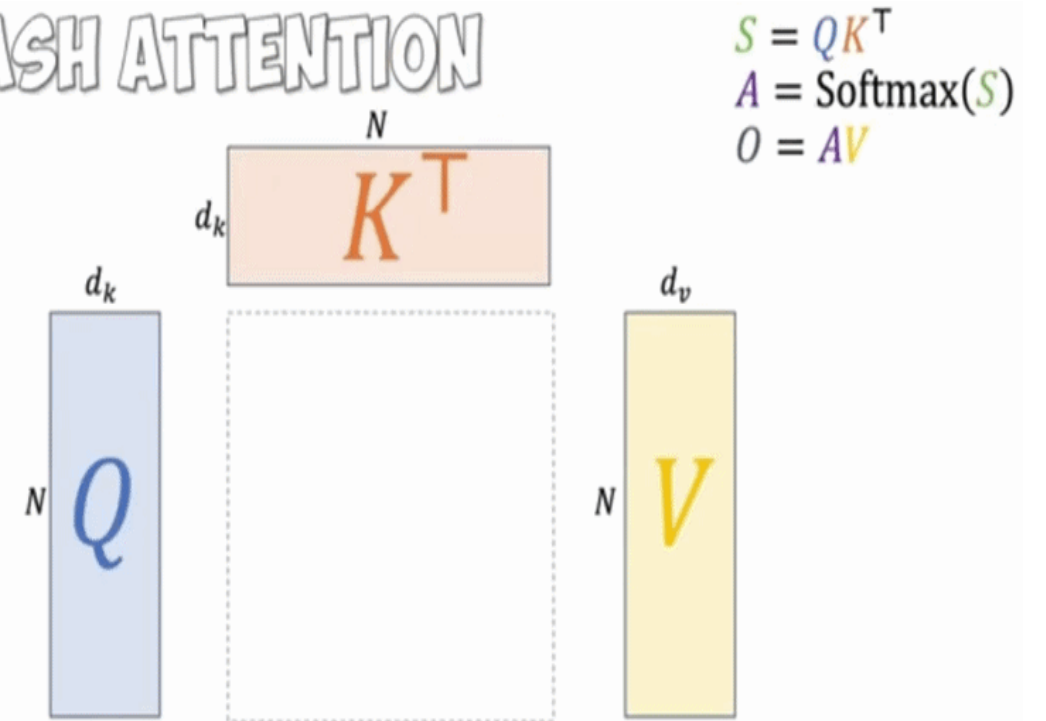
$$A = \text{Softmax}(S)$$

Attention weight

Flash Attention



FLASH ATTENTION



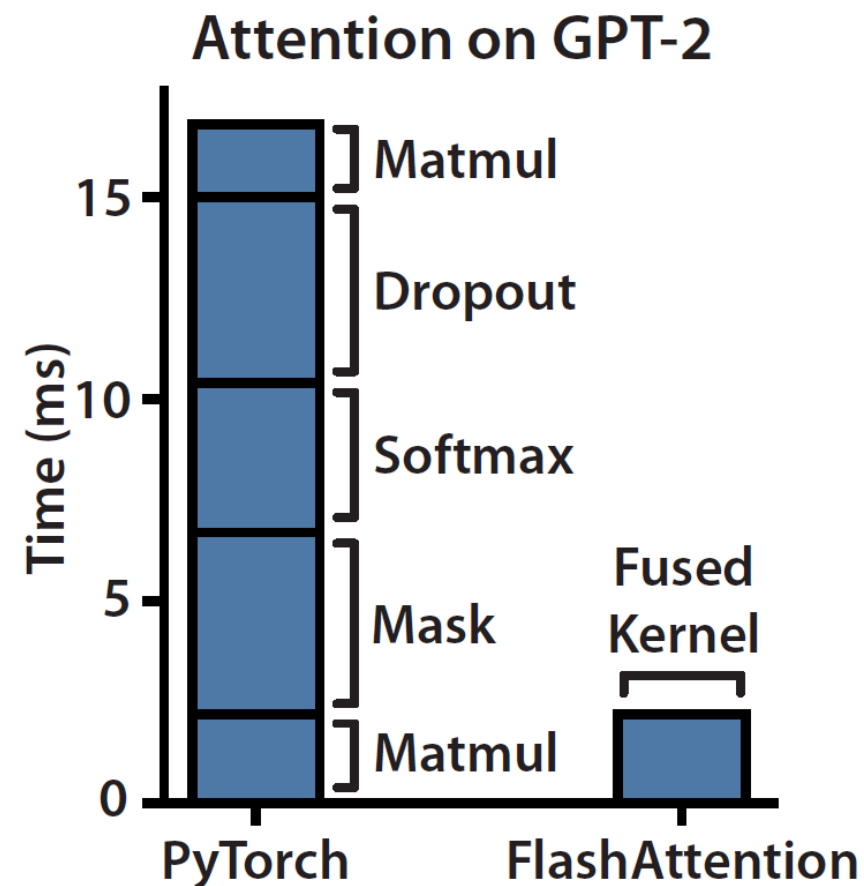
$$S = QK^T$$

$$A = \text{Softmax}(S)$$

$$O = AV$$

Flash Attention

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	2.7 days (3.5×)
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	6.9 days (3.0×)



Thank You